

ESP32 Display Development Log

Matthew Sylvester

January 2026 – current

1 Resources

- entire module wiki
- RAM module specsheet

2 Gif converter

Uses typescript frontend webapp to convert gif files into hex for upload to ESP.

2.1 Libraries

2.1.1 gifuct-js

Allows for parsing and decompressing of gif file.

2.2 functions

2.2.1 loadGIF

- input: Gif file
- output: frames[], each frame in frames is an array which looks like { width, height, delay, patch (Uint8ClampedArray (RGBA)) }

intakes gif & converts it into an array of frames w/ frame data

2.2.2 frameToImageData

- input: single frames[] array element
- output: imageData object

draws the current frame to a canvas

2.2.3 rgbTo565

- input: single rgb pixel
- output: 16 bit hex value of the pixel

3 ESP32 Deployment

3.1 Notes

Strange behaviour with colours, green and blue are swapped. I think there's some sort of strangeness with how the colours are sent over SPI, or with the driver somewhere. The normal RGB565 format is RRRRRGGGGGGBBBBB, but the display seems to be displaying as RRRRRBBBBBGGGGGG, or maybe RRRRRBBBBBGGGGGG. Bitwise shifting the blue value to the left by 6 bits and green to the right by 5 bits resolves the issue but it's a bit of a hacky solution that loses us 1 bit of precision in green. The function `uint16_t colorfix(uint16_t color)` handles shifting any colours.

Resolved with `lv_draw_sw_rgb565_swap(src, w * h)`! See explanation here: https://docs.lvgl.io/9.4/API/draw/sw/lv_draw_sw_utils_h.html#_CPPv422lv_draw_sw_rgb565_swapPv8uint32_t Have not yet tested outside of lvgl drawing, but we'll go with this for now, as we shouldn't need to operate outside of lvgl.

Save images to PSRAM (2mb limit). This means images are lost on power disconnect.

Monitor ADC for battery voltage and go into deep sleep when battery is low?

3.2 Peripheral Functions

3.2.1 Backlight Brightness Control

Controlled with a queue and handler task. The handler waits until it sees a message in the queue, then sets the brightness to the requested duty cycle. Duty cycle resolution is 8 bits, so pass a value between 0 and 255. Include `display.h` to use the queue, then send messages like this:

```
bl_msg_t msg = {  
    .duty = 128  
};
```

Then send the message to the queue with `xQueueSend(bl_queue, &msg, portMAX_DELAY);`.